

Informe Técnico — Proyecto ITACA

Sistema Inteligente de Autodiagnóstico Empresarial (SIAE)

Versión: 2.0.0 · Fecha: Agosto de 2026 · Stack: TensorFlow/Keras · SentenceTransformers · FastAPI · Docker

1. Descripción y Justificación del Problema

1.1 Contexto de negocio

Ítaca acompaña a pequeñas y medianas empresas en su proceso de transformación digital. El primer paso de ese acompañamiento — diagnosticar el nivel de madurez digital de cada empresa — se realizaba de forma manual: un consultor entrevistaba a la organización, revisaba sus procesos y emitía un dictamen. Este enfoque presenta tres limitaciones estratégicas:

- **Escalabilidad:** el diagnóstico manual toma horas por empresa; la demanda supera la capacidad del equipo consultor.
- **Consistencia:** dos consultores pueden emitir dictámenes distintos ante evidencia similar, introduciendo subjetividad.
- **Costo de entrada:** para una microempresa, el costo de una consultoría inicial puede ser disuasorio, dejando fuera al segmento que más la necesita.

1.2 ¿Por qué IA/Deep Learning y no métodos tradicionales?

La evidencia disponible para el diagnóstico es inherentemente **multimodal**: combina variables estructuradas (sector, tamaño, presupuesto tecnológico) con texto libre donde la empresa describe sus procesos. Un sistema de reglas o un modelo tabular clásico puede tratar las variables estructuradas, pero no puede interpretar frases como “*el trabajo es muy empírico, dependemos de registros en papel*” — que es donde reside la señal más rica del diagnóstico. El aprendizaje profundo permite fusionar ambas modalidades en una única red con dos ramas de entrada, aprovechar transfer learning en la rama de texto y escalar a coste marginal cercano a cero: una vez entrenado, cada diagnóstico adicional cuesta aproximadamente 0,3 segundos de cómputo.

2. Diseño Técnico y Metodología

2.1 Arquitectura multi-input

El modelo es una red Keras funcional con dos ramas de entrada que se fusionan antes de la capa de clasificación. La **rama tabular** recibe 9 features (codificación one-hot de sector y tamaño de empresa, más presupuesto escalado). La **rama de texto** recibe el embedding de 384 dimensiones de la respuesta textual. Ambas pasan por capas densas con BatchNormalization, se concatenan, y una capa softmax produce la distribución sobre las 4 clases de madurez: Inicial, En Desarrollo, Definido y Optimizado.

Componente	Detalle
Entrada tabular	9 features (one-hot + escalado)
Entrada de texto	Embedding 384d (SentenceTransformer)
Fusión	Concatenate + capas densas

Salida	Softmax, 4 clases
Entrenamiento	30 épocas, batch 32, pesos de clase balanceados

2.2 Justificación de TensorFlow/Keras

La API funcional de Keras expresa arquitecturas multi-input de forma declarativa y legible. El formato de serialización .keras empaqueta arquitectura y pesos en un único artefacto portable, cargado en producción por la API. El ecosistema ofrece ejecución CPU-only eficiente (tensorflow-cpu), relevante porque el despliegue objetivo (Docker en infraestructura de PyME) no dispone de GPU.

2.3 Estrategia de Transfer Learning

Entrenar un modelo de lenguaje desde cero requeriría millones de documentos. En su lugar, la rama de texto usa SentenceTransformers con el modelo preentrenado **paraphrase-multilingual-MiniLM-L12-v2**: multilingüe (incluye español), ligero (~120 MB, apto para contenedores sin GPU) y productor de embeddings de 384 dimensiones que capturan similitud semántica. El transformer actúa como extractor de características congelado: solo se entrenan las capas densas posteriores, lo que reduce el entrenamiento completo a menos de 2 minutos en CPU.

2.4 Pipeline de entrenamiento

El script scripts/train.py ejecuta el flujo end-to-end de forma reproducible: (1) partición estratificada 70/15/15; (2) ajuste del preprocesador tabular solo con train; (3) limpieza NLP y codificación con el transformer; (4) entrenamiento; (5) evaluación sobre test con auditoría de equidad por sector; (6) exportación de artefactos: model.keras, tabular_preprocessor.joblib, label_encoder.joblib, metadata.json y metrics_report.json.

3. Especificación de Datos

Aspecto	Detalle
Volumen	3.000 registros
Formato	CSV (UTF-8), 8 columnas
Variables tabulares	sector (4), tamano_empresa (4), porcentaje_procesos_documentados, presupuesto_anual_tecnología
Variable textual	respuesta_texto (descripción libre de procesos)
Etiqueta	nivel_madurez (4 clases)
Distribución	Inicial 26% · En Desarrollo 31% · Definido 30% · Optimizado 12%

3.1 Calidad y limitaciones detectadas

El análisis exploratorio y los scripts de auditoría revelaron dos problemas críticos de calidad, tratados en detalle en la sección 5: (1) una fuga determinista en la variable porcentaje_procesos_documentados, que determina la etiqueta con exactitud del 100% y fue excluida del entrenamiento; y (2) un corpus textual sintético de baja variedad: solo 32 textos únicos en 3.000 filas, cada uno asociado a exactamente una clase.

3.2 Plan de preprocesamiento

Tabular: codificación one-hot de categóricas y escalado de numéricas, encapsulado en un TabularPreprocessor serializable que garantiza transformaciones idénticas en entrenamiento y producción. **Texto:** normalización a minúsculas, eliminación de stopwords en español y codificación con SentenceTransformer. **Particiones:** estratificadas por etiqueta.

4. Análisis Ético y de Responsabilidad

4.1 Anonimización y PII

El dataset de trabajo es sintético: no contiene datos de empresas reales, por lo que el riesgo de exposición de PII en esta fase es nulo. El diseño anticipa el paso a datos reales: el identificador id_diagnostico es un código opaco; no se recogen nombres, NIT/RUC, direcciones ni contactos; y el texto libre será sometido a un filtro NER antes de almacenarse para eliminar menciones accidentales de personas u organizaciones. Los artefactos serializados no contienen registros del dataset.

4.2 Auditoría de equidad (Fairness)

La equidad entre sectores es un requisito de diseño verificado en cada entrenamiento por el componente ITACAModelEvaluator: calcula el F1 por sector y verifica que la diferencia máxima entre sectores no supere el 5% (umbral max_fairness_delta = 0.05). El resultado queda registrado en artifacts/metrics_report.json con veredicto explícito. En la ejecución actual: F1 = 1.0 en los cuatro sectores, delta = 0.0. **Lectura crítica:** dado el problema de memorización descrito en la sección 5, este resultado confirma que el mecanismo de auditoría funciona, pero no constituye aún evidencia de equidad en condiciones reales.

4.3 Estrategias de explicabilidad (XAI)

- **Estudios de ablación** (ablacion_solo_texto.py): aíslan la contribución de cada modalidad al diagnóstico.

- **Diagnóstico de fuga** (`diagnostico_fuga.py`): verifica con un árbol trivial si una variable individual determina la etiqueta — técnica que detectó la fuga documentada.
- **Transparencia de incertidumbre**: la API devuelve la distribución completa de probabilidades y el frontend alerta cuando la confianza es menor a 0.55, recomendando revisión humana.
- **Trabajo futuro**: importancia por permutación sobre features tabulares y análisis de similitud de embeddings.

4.4 Alcance del sistema

El sistema se presenta explícitamente como orientativo: la interfaz muestra de forma permanente la nota “Este diagnóstico es orientativo y no reemplaza una consultoría profesional”, y los casos de baja confianza se derivan a revisión humana. El modelo recomienda; no decide.

5. Resultados y Conclusiones

5.1 Métricas formales frente a metas

Meta	Objetivo	Resultado formal	¿Cumple?
Accuracy	$\geq 85\%$	100%	Sí *
F1-macro	≥ 0.80	1.00	Sí *
Equidad (delta F1 sectores)	$\leq 5\%$	0.0%	Sí *
Latencia (inferencia caliente)	$< 3 \text{ s}$	0.30 - 0.32 s	Sí
Despliegue reproducible	end-to-end	Docker Compose funcional	Sí

* Con la salvedad crítica que se detalla a continuación.

5.2 Análisis crítico: por qué las métricas perfectas son una señal de alerta

Un accuracy de 1.0 en un problema real de clasificación multimodal es estadísticamente implausible. El equipo trató este resultado como un síntoma a investigar, no como un éxito, y la investigación produjo los dos hallazgos centrales del proyecto.

Hallazgo 1 — Fuga en variable numérica (corregida). porcentaje_procesos_documentados determina la etiqueta de forma exacta: un árbol de decisión de profundidad 4 entrenado solo con esa columna alcanza accuracy 1.0. La variable fue excluida del entrenamiento.

Hallazgo 2 — Memorización del corpus textual (estructural). Tras excluir la variable con fuga, el accuracy se mantuvo en 1.0. La causa: el corpus contiene únicamente 32 textos únicos para 3.000 filas (8 por clase), cada texto pertenece exactamente a una clase, y el 100% de los textos de test aparecen literalmente en train. El modelo no aprende a interpretar lenguaje: memoriza 32 cadenas. Las métricas miden capacidad de memorización, no de generalización.

Implicación metodológica: una métrica honesta de generalización requiere (a) partición agrupada por texto (GroupShuffleSplit) y, de forma más fundamental, (b) un corpus con variedad textual real. La recolección de respuestas textuales reales es el paso crítico previo a cualquier afirmación de desempeño.

5.3 Nivel de madurez tecnológica (TRL)

El sistema alcanza **TRL 5-6**: prototipo integrado y validado en un entorno relevante (contenedores, API real, cliente web, latencia verificada). La transición a TRL 7 (demostración en entorno operativo) queda condicionada a la validación con datos de empresas reales.

5.4 Conclusiones

1. La ingeniería del sistema está completa y es sólida: pipeline reproducible, artefactos versionados, API con manejo seguro de errores, auditoría de equidad automatizada, frontend con dashboard comparativo y despliegue contenedorizado con healthchecks.
2. El hallazgo más valioso del proyecto es metodológico: la detección y documentación de dos fugas de datos encadenadas demuestra madurez en auditoría de ML — una competencia más escasa y valiosa que reportar métricas altas sin escrutinio.
3. El camino a producción está claramente definido: recolectar corpus textual real, re-particionar por grupos, re-entrenar y re-auditar equidad. La arquitectura no requiere cambios para ese paso; solo los datos.

